- `Name: Digit`
- `Age: 17`

We have already seen an instance of default argument in the `fifa_winner` function. In this case, there is one instance of keyword definition given already in the function definition. In the case of `fifa_winner,` it's `country="England"`. So unless you specifically define the country, it will print the default argument. There might be multiple arguments given in the function with a combination of defined and undefined. E.g. amending the `fifa_winner` function to

- `def fifa _ winner(country="England", year="2018"):`
- `print ("I want " + country + " to win the " + year + " FIFA cup")`

Some combinations in which we can call this code and its outputs can be seen here: *https://dgit.in/KeyFuncPy.* If you were to define only one variable like

- `def fifa _ winner(year, country="England"):`
- `print ("I want " + country + " to win the " + year + " FIFA cup")`

You will have to define the variable "year" when calling the function since there is no default value given for it. So if you want call the function retaining the available default value, it will read as

- `fifa _ winner(year="2022")`

The resulting output will be

- `I want England to win the 2022 FIFA cup`

using the default value for the country variable and the defined value used while calling. If no keyword is defined here while calling the function, it will give the error- `TypeError: fifa_winner() missing 1 required positional argument: 'year'.` One point to remember while building this function is that the variable parameter always has to come before the defined parameter in the function definition. So it always has to be `(year, country="England")` and never `(country="England", year)`. Using this will give the error- `SyntaxError: non-default argument follows default argument.`

The last type is the variable length arguments wherein the length of argument is not fixed. So it could have one, two or more input values. The variable argument is prefixed with an asterix (*). Take a look at the code here: *https://dgit.in/VarLenPy.* When emulating this type of code, follow the exact syntax with the indentations and colons